

Capstone Project Playbook

From idea to a working demo — and how it is marked

1. How to use this

This playbook works for **any** capstone project — one from the project catalogue, or one you propose yourself.

Your work moves through **six phases**. You move to the next phase when you pass the current one's exit check, not when a date arrives. Some phases will take you longer than others; that is expected.

The marks tell you where the effort belongs. Phases 1–4 are worth 20 each, Phases 5–6 are worth 10 each. Everything you are graded on is listed in §4 — nothing is hidden.

2. Choosing your own project

Bringing your own idea? Put it through four tests first. If it fails even one, change the idea now — not in Phase 4, when it is too late.

Test 1 — Is it a real problem? Name the person who has this problem and what it costs them today: hours lost, money wasted, mistakes made. "It would be cool" is not a problem.

Test 2 — Is there one core feature? Write this sentence: "You give it ___, it gives you ___." If you need three sentences, your scope is too big. Cut it until one sentence covers it.

Test 3 — Can you get real data? Find the actual dataset, API, or documents **before** you commit. Projects collapse in week two when the data turns out to be private, paid, or nonexistent.

Test 4 — Can you demo it in 5 minutes? Picture the final demo. If it needs a long setup, or the result is invisible, choose something you can show.

Passed all four? Write a one-page proposal — problem, the one-sentence core feature, data source, stack — and get it approved before you start Phase 2.

3. The six phases

Phase	What you produce	Marks
1 · Scope & Design	Project document + architecture diagram	20
2 · Skeleton	A repo that runs end to end, roughly	20
3 · Core Feature	The one thing your project exists to do	20
4 · End-to-End	One flow that survives being used	20
5 · Polish	An interface a stranger can drive	10
6 · Demo & Defence	A live demo and your answers	10

Phase 1 — Scope & Design · 20 marks

Goal: agree what you are building, how the pieces fit, and who owns what.

Exit check — move on only when all four are true:

- the problem is stated with a number attached — hours lost, cost, or error rate
- scope is bounded **in writing**, including a list of what you will *not* build
- an architecture diagram shows the main components and how data moves between them
- every member has a named area they own

Graded on: Clarity of problem (5) · Feasible scope (5) · Architecture understanding (5) · Team roles defined (5)

Phase 2 — Skeleton · 20 marks

Goal: get a thin path running from one end to the other. Ugly is fine — working is the point.

Exit check:

- anyone can clone the repo and run it without dependency errors
- your external service — API, model, or database — returns **real** data, not a stub
- every member has pushed at least one meaningful commit
- you can show it running live, even if the output is raw text in a console

Graded on: Code runs (5) · API/LLM integration (5) · Repo + commits (5) · Live demo success (5)

Phase 3 — Core Feature · 20 marks

Goal: the one thing your project exists to do, working on real data.

Exit check:

- the core feature runs on real data, not samples you wrote yourself
- you have 3 test inputs you can run on demand, in front of anyone
- what you have built still matches the scope you set in Phase 1

Graded on: Core feature implementation (8) · Real data usage (4) · Test cases (4) · Progress vs plan (4)

Phase 4 — End-to-End · 20 marks

Goal: connect every part into one flow, then make it survive contact with a real user.

Exit check:

- one input travels the whole system and comes back as a finished result
- it does not crash when something goes wrong — a slow API, a bad file, an empty result
- the output is correct and honest: no invented facts, no fake citations
- you keep a short log of bugs you hit and how you fixed them

Graded on: End-to-end integration (8) · Stability, no crashes (4) · Output quality (4) · Debugging effort (4)

Phase 5 — Polish · 10 marks

Goal: make it understandable to someone who has never seen it before.

Exit check:

- a first-time user knows what to do without being told
- the app shows progress while it works — nothing ever looks frozen
- every feature you scoped in Phase 1 is present, or you can say why it is not
- the demo is pre-loaded and has been run end to end

Graded on: UI clarity (3) · Demo readiness (3) · Storytelling (2) · Feature completeness (2)

Phase 6 — Demo & Defence · 10 marks

Goal: show it working, and defend the choices you made.

Exit check — prepare all four before you present:

- the demo runs live, and you have a recorded video as backup
- you can explain why you chose your approach over the obvious alternative
- every member speaks and owns a part of the demo
- you can name your project's weakest point before anyone asks you

Graded on: Demo success (4) · Technical understanding (3) · Team participation (2) · Q&A handling (1)

4. How you are marked

100 marks across six phases. Same criteria for every project.

Phase	Criteria	Marks
1 · Scope & Design	Clarity of problem	5
	Feasible scope	5
	Architecture understanding	5
	Team roles defined	5
2 · Skeleton	Code runs	5
	API/LLM integration	5
	Repo + commits	5
	Live demo success	5
3 · Core Feature	Core feature implementation	8
	Real data usage	4
	Test cases	4
	Progress vs plan	4
4 · End-to-End	End-to-end integration	8
	Stability (no crashes)	4
	Output quality	4
	Debugging effort	4

Phase	Criteria	Marks
5 • Polish	UI clarity	3
	Demo readiness	3
	Storytelling	2
	Feature completeness	2
6 • Demo & Defence	Demo success	4
	Technical understanding	3
	Team participation	2
	Q&A handling	1
	Total	100

5. Submission checklist

- ☐ Public GitHub repo with a README saying what it does and how to run it
- ☐ Setup instructions someone outside your team has actually followed
- ☐ Your Phase 1 project document, updated to match what you really built
- ☐ 3 test inputs with their expected outputs
- ☐ Presentation deck
- ☐ Backup demo video, 2–3 minutes
- ☐ Deployed link, if you have one

6. Common failure modes

Scope grows quietly. Every feature you add is a feature you will not finish. Adding one is a team decision, written down.

Integration is left until the end. Parts that work alone rarely work together on the first try. Connect them in Phase 2, even while each one is still a stub.

The demo runs on dummy data. It proves nothing, and evaluators spot it immediately. Get real data flowing in Phase 2.

One person codes, the rest watch. Commits and team participation both carry marks. Split the work by component, not by who is fastest.

The demo is rehearsed once, the night before. Run it end to end at least three times, on the machine you will actually present from.

Nobody can explain the design. You will be asked why you chose your approach. Decide it together, and make sure every member can answer.

7. Team roles

Every member owns an area and can explain it. A typical split for a four-person team:

Role	Owns
Data & Integration	Sources, APIs, cleaning, the pipeline feeding the core
Core Logic	The model, agent, or algorithm doing the real work
Interface	How a user drives it and sees results
Quality & Demo	Tests, error handling, the deck, the demo script

Smaller teams combine roles — but nobody holds zero. Everyone commits code, and everyone speaks at the demo.